

Topological image feature extraction for Visual Transformer Breast Cancer Clas- sification



Universitat Oberta
de Catalunya

Esther Ruano Hortonedá

Machine learning in clinical data analytics

Master in Data Science

Name of the tutor:

Ana Belén Nieto Librero

Name of the SRP:

Ana Belén Nieto Librero

May 8, 2026



This license lets others distribute, remix, adapt, and build upon your work, even commercially, as long as they credit you for the original creation. This is the most accommodating of licenses offered. Recommended for maximum dissemination and use of licensed materials.

<https://creativecommons.org/licenses/by-nc/3.0>

Final Work Card

Title of the work:	Topological image feature extraction for Visual Transformer Breast Cancer Classification
Name of the author:	Esther Ruano Hortonedá
Name of the tutor:	Ana Belén Nieto Librero
Name of the SRP:	Ana Belén Nieto Librero
Date of delivery:	May 8, 2026
Studies or Prgram:	Master in Data Science
Area or the Final Work:	Machine learning in clinical data analytics
Language of the work:	English
Keywords:	Image classification, Deep learning, Breast cancer

Abstract

This project aims to develop a Deep Learning pipeline for the binary classification of mammographic images (tumor vs. non-tumor) using the CBIS-DDSM dataset. The proposed approach combines image preprocessing techniques, including noise reduction and contrast enhancement, with feature extraction based on Topological Data Analysis (TDA) to capture structural properties of breast tissue.

Both the processed images and the extracted topological descriptors will be integrated with a Vision Transformer (ViT) model for final classification. The objective is to evaluate whether incorporating explicit topological information improves classification performance.

Contents

1	Introduction	8
1.1	Context and justification of work	8
1.2	Work objectives	9
1.3	Impact on sustainability, ethics-social and diversity	10
1.4	Focus and method followed	10
1.4.1	Requirement Analysis	11
1.5	Work schedule	11
1.6	Brief summary of products obtained	12
1.7	Brief description of other memory chapters	12
2	State of Art	13
2.1	CBIS-DDSM dataset	13
2.1.1	Evaluation Metrics	14
2.1.2	Studies done with the CBIS-DDSM Dataset	15
2.2	Visual Transformers (ViT)	17
2.3	Topological Data Analysis (TDA) applied to images	20
2.4	Combining ViT and TDA:	22
3	Materials and methods	23
3.1	Data preparation	23
3.1.1	Dataset selection and scope	23
3.1.2	Data loading and file resolution	23
3.1.3	Train, validation, and test splits	24
3.1.4	Image caching	24
3.2	Image preprocessing	24
3.3	Topological feature extraction	25
3.3.1	Persistence images	25
3.3.2	Compact vector descriptors	26
3.4	Model architectures	26
3.4.1	ViT classifier	26
3.4.2	Dual-ViT fusion model	27
3.4.3	Linear fusion model	28
3.5	Data augmentation	28
3.6	Training protocol	28
3.7	Experimental design	31

3.7.1	Hypothesis 1: Impact of preprocessing (E1–E3)	32
3.7.2	Hypothesis 2: Impact of TDA features (E4–E7)	32
3.8	Implementation and reproducibility	33
4	Statement on the use of generative artificial intelligence	38
5	Results	39
5.1	Hypothesis 1: Effect of preprocessing	39
5.2	Hypothesis 2: Effect of TDA features	40
5.3	Learning curves	41
5.4	Confusion matrices and ROC curve	42
5.5	Comparison with the literature	43
6	Discussion	44
7	Conclusions and future works	45
8	Glossary	46

List of Figures

1.1	Gantt diagram	12
2.1	Residual Network and Visual Transformer Architectures	19
2.2	How to compute a persistence diagram	21
3.1	Effect of the preprocessing pipeline on a sample cropped mammogram: (a) original image, (b) after CLAHE, (c) after CLAHE + Gaussian denoise.	25
3.2	ViT classifier architecture: the grayscale mammogram is replicated to 3 channels, resized to 224×224 , and passed through the pretrained ViT-B/16. The CLS token is projected to a single logit via a linear layer.	27
3.3	Dual-ViT fusion architecture: the mammogram and the persistence image are each processed by a separate pretrained ViT-B/16. The two CLS token embeddings (768-dim each) are concatenated and classified by an MLP.	29
3.4	Linear fusion architecture: the mammogram is processed by ViT-B/16 (768-dim CLS token), while the 1400-dim TDA vector is projected to 128 dimensions. Both are concatenated and classified by an MLP.	30
5.1	AUC, Accuracy, and F1 comparison for experiments E1–E3 (Hypothesis 1). . . .	40
5.2	AUC, Accuracy, and F1 comparison for experiments E4–E7 (Hypothesis 2) against the E2 baseline (dashed line).	40
5.3	Training loss (blue), validation loss (orange) and validation AUC across epochs for all experiments. Early stopping is indicated by the curve termination point. .	41
5.4	Confusion matrices for each experiment	42
5.5	ROC curve for each experiment	42

List of Tables

2.1	Summary of CBIS-DDSM binary classification studies	17
3.1	Training hyperparameters	31
3.2	Summary of experimental configurations	32
5.1	Test set results for all experiments	39
5.2	Comparison with CBIS-DDSM binary classification studies from the literature .	43

Chapter 1

Introduction

Breast cancer remains the most prevalent oncological disease worldwide. According to the World Health Organization [1], it represents a leading cause of cancer-related mortality among women globally. Early detection plays a crucial role in improving survival rates, and medical imaging techniques such as mammography are fundamental tools in preliminary diagnosis.

In recent years, significant advances in Artificial Intelligence and Deep Learning have transformed medical image analysis. New approaches such as lightweight architectures [2], hybrid feature extraction techniques [3], and transfer learning strategies [4] have demonstrated promising results in improving diagnostic performance while reducing computational cost. These developments open new possibilities for enhancing automated breast cancer detection systems.

Motivated by these advances and supported by a strong mathematical and analytical background, this work explores the integration of modern deep learning architectures with advanced image processing and structural feature extraction techniques for mammographic image classification.

1.1 Context and justification of work

Breast cancer diagnosis has become an active research area due to its medical relevance and the rapid development of computational methods for medical imaging analysis. The current research landscape shows a significant growth in the application of deep learning techniques for tumor detection and classification, including generative adversarial networks (GANs), convolutional neural networks (CNNs) and large language models (LLMs) [5].

Breast imaging can be obtained through several procedures: mammography, ultrasound, magnetic resonance imaging (MRI), synthetic phantoms, etc. Among publicly available datasets, two widely used resources are the Curated Breast Imaging Subset of Digital Database for Screening Mammography (CBIS-DDSM) [6] and the VICTRE breast phantom dataset [7]. Since mammography remains the most common imaging modality for preliminary breast cancer screening, this work focuses on the CBIS-DDSM dataset to ensure clinical relevance and comparability with previous studies.

Image preprocessing plays a critical role in image classification. Common techniques include noise reduction and contrast enhancement, which improves the visibility of relevant anatomical structures and lesions. Previous studies have demonstrated the effectiveness of such preprocessing steps in improving classification performance [8, 9]. Therefore, similar preprocessing

strategies are incorporated into the proposed pipeline.

In addition to conventional preprocessing, this work introduces Topological Data Analysis (TDA) as a complementary feature extraction approach. Recent research has shown promising results when integrating topological information into deep learning models [10]. TDA captures structural and connectivity-related properties of images that may not be fully represented by pixel-based learning alone. Furthermore, the use of explicit structural descriptors may contribute to improved explainability, which is particularly important in medical applications.

Regarding model selection, comparative studies evaluating different deep learning architectures for breast cancer classification indicate that Vision Transformers (ViT) achieve competitive or superior performance compared to traditional convolutional networks [11, 12]. Based on these findings, the Vision Transformer architecture is adopted as the primary classification model in this work.

1.2 Work objectives

The central hypothesis of this work is that the integration of image preprocessing techniques and Topological Data Analysis (TDA) features into a Vision Transformer (ViT) architecture improves the classification performance of mammographic images compared to a baseline ViT model without such enhancements.

More specifically:

- H1: Image preprocessing techniques (noise reduction and contrast enhancement) positively impact the classification accuracy of a Vision Transformer model.
- H2: The incorporation of topological descriptors derived from TDA provides complementary structural information that improves classification performance compared to a standard Vision Transformer.

To test these hypotheses, the following objectives are defined:

- To implement a baseline Vision Transformer model for mammographic image classification using the CBIS-DDSM dataset.
- To evaluate the impact of image preprocessing techniques on the performance of the baseline model.
- To extract topological features from mammographic images using Topological Data Analysis methods.
- To design the architecture integrating TDA features with the Vision Transformer model.
- To compare all approaches using quantitative performance metrics such as accuracy, sensitivity, specificity, F1-score, and AUC.
- To develop a theoretical and practical understanding of Vision Transformer architectures and medical image preprocessing techniques.

1.3 Impact on sustainability, ethics-social and diversity

This project is aligned with the United Nations Sustainable Development Goals (SDGs) [13], particularly Goal 3 (Good Health and Well-being) and Goal 10 (Reduced Inequalities).

Breast cancer is one of the leading causes of cancer-related mortality worldwide [1]. Early diagnosis and treatment through screening programs significantly increases survival rates. The development of computer-aided diagnostic tools based on Artificial Intelligence can support medical professionals by assisting in the early detection of suspicious lesions, potentially reducing diagnostic delays and improving patient outcomes (SDG 3).

From a social perspective, automated diagnostic support systems may help reduce inequalities in access to specialized medical expertise (SDG 10). In regions with limited access to experienced radiologists, AI-based tools could provide preliminary assessments, contributing to more equitable healthcare delivery.

However, the ethical implications of AI in medical decision-making must be carefully considered. Automated systems should not replace medical professionals but rather support them. Issues such as model bias, generalization across populations, transparency, and interpretability are critical. In this work, the incorporation of Topological Data Analysis aims not only to improve performance but also to enhance structural interpretability of the learned features.

Regarding sustainability, the computational cost of training deep learning models is also considered. The use of efficient architectures and controlled experimentation seeks to balance model performance with responsible resource consumption.

1.4 Focus and method followed

This work follows an experimental research strategy within the field of applied machine learning. The objective is to design, implement, and evaluate a deep learning-based classification system for mammographic images.

The research adopts a quantitative approach, where performance metrics obtained from experiments are used to validate the proposed hypotheses. Since the study focuses on computational model evaluation using publicly available datasets, no qualitative data collection methods are involved.

From a development perspective, this project can be classified as the design and implementation of a new computational pipeline integrating existing technologies (Vision Transformers and Topological Data Analysis) into a novel hybrid architecture.

The methodological process consists of the following stages:

1. Literature review on breast cancer image classification, preprocessing techniques, transformer-based architectures, and TDA.
2. Dataset selection and preparation using the CBIS-DDSM dataset.
3. Implementation of a baseline Vision Transformer model.
4. Integration of preprocessing techniques (noise reduction and contrast enhancement).
5. Extraction of topological descriptors and design of a hybrid model architecture.

6. Experimental evaluation and comparison of different model configurations using quantitative metrics.
7. Statistical analysis and interpretation of results.

The project follows an iterative prototyping methodology. Models are developed incrementally, starting from a baseline implementation and progressively integrating preprocessing and topological features. Each iteration includes training, validation, performance evaluation, and refinement.

The tools used include programming environments for deep learning model development, numerical computation libraries, and image processing frameworks appropriate for medical image analysis. In addition, a requirement analysis is conducted to clearly define the system specifications and guide the development process.

1.4.1 Requirement Analysis

The project requires the definition of both functional and non-functional requirements to ensure a structured development process.

Functional requirements:

- Load and preprocess the CBIS-DDSM dataset.
- Implement a Vision Transformer (ViT) for binary classification (benign vs malignant).
- Extract and integrate Topological Data Analysis (TDA) features.
- Train, validate, and test the model.
- Evaluate performance using metrics such as accuracy, precision, recall, and F1-score.

Non-functional requirements:

- Reproducibility of experiments.
- Computational efficiency.
- Modularity of the implementation.
- Interpretability of results.

1.5 Work schedule

The work schedule for this project has been structured to align with the defined objectives and deliverables. A Gantt diagram 1.1 has been created to break down each deliverable into individual tasks, illustrating their planned start and end dates, dependencies, and expected duration. The schedule is periodically updated to reflect the current progress, with the latest update recorded as of March 22nd, 2026.

This structured plan ensures systematic progress toward the project goals, facilitates monitoring of milestones, and allows timely adjustments if necessary.

GANTT CHART

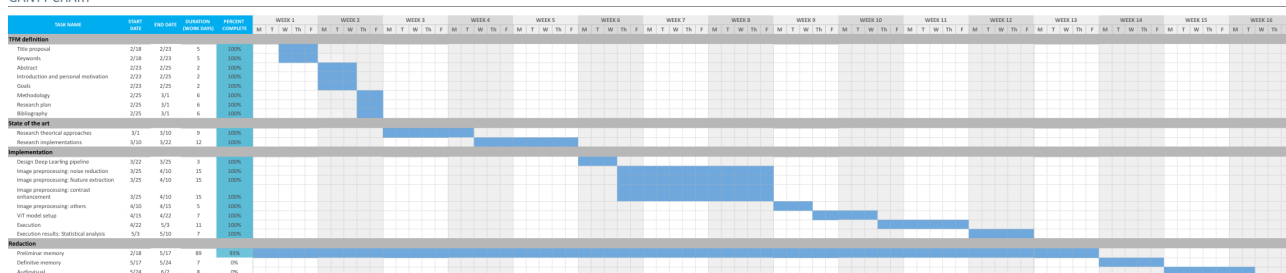


Figure 1.1: Gantt diagram

1.6 Brief summary of products obtained

No need to go into detail: the detailed description will be made in the rest of the chapters.

1.7 Brief description of other memory chapters

Brief explanation of the contents of each chapter and its relationship with the global project.

Chapter 2

State of Art

In this chapter we will cover the state of the Art of each of the components involved in the model proposed in the Context and justification of work 1.1 above. This chapter is split into four sections:

1. **How CBIS-DDSM is used and evaluated in the literature:** describes the characteristics of the dataset and how the dataset has been studied. It also explores what techniques and metrics are used to evaluate the results and their meaning.
2. **Visual Transformers (ViT):** explains what visual transformers are, and compares them to classical Convolutional Neural Networks (CNNs).
3. **Topological Data Analysis (TDA) applied to images:** details what topological features are and how they can be extracted from images.
4. **Combining ViT and TDA:** explores the research gap and how this thesis can help reduce it.

Following these steps, it will provide a comprehensive review of the current status of the research related to the project.

2.1 CBIS-DDSM dataset

In this section we will explore how the CBIS-DDSM dataset is used and evaluated in the literature. First, let's recall the characteristics of the dataset: by visiting the documentation provided by the Cancer Imaging Archive [6], the DDSM is a database of 2,620 scanned film mammography studies. It contains normal, benign, and malignant cases with verified pathology information from 1566 subjects. Details on how to use the dataset were published simultaneously [14], and it addresses a key limitation in previous research: the lack of standardized and reproducible evaluation protocols in mammography-based computer-aided diagnosis systems. Prior studies often relied on private datasets or non-standard subsets of public databases, making direct comparison between methods difficult. To overcome this issue, the authors introduced a curated and standardized version of the original dataset: decompressed images in DICOM

format, improved region-of-interest (ROI) annotations, and verified pathology labels. Additionally, predefined training and testing splits were provided to facilitate consistent evaluation across different studies. Another important contribution of this work is the enhancement of annotation quality. The ROI segmentations were refined using automated methods and validated against expert radiologist annotations using metrics such as the Dice coefficient, showing a significant improvement over the original DDSM annotations. This increases the reliability of the dataset as a benchmark for both detection and segmentation tasks. Overall, CBIS-DDSM was designed not only as a larger and cleaner dataset, but as a step towards more reproducible and comparable research in breast cancer diagnosis using medical imaging.

This dataset has been used to study breast cancer from very different angles: from classification problems (binary [15]) or multi-class [16]), localization [17], segmentation [18], etc.

In this paper, we will focus on binary classification; in the next paragraphs we visit different binary classification relevant papers that have used this dataset, what results have they obtained and what do these results mean. Before doing that, we cover some definitions regarding binary classification evaluation [15].

2.1.1 Evaluation Metrics

The metrics used in the papers that we have studied are the following:

Confusion Matrix

A confusion matrix is a tabular representation that is used to analyse the performance of a classification model by comparing predicted labels with the true labels. In the case of binary classification, the confusion matrix consists of four main components:

- True Positive (TP): the number of instances that belong to the positive class and are correctly classified as positive.
- True Negative (TN): the number of instances that belong to the negative class and are correctly classified as negative.
- False Positive (FP): the number of instances that belong to the negative class but are incorrectly classified as positive.
- False Negative (FN): the number of instances that belong to the positive class but are incorrectly classified as negative.

Accuracy

Accuracy is one of the most commonly used evaluation metrics. It measures the proportion of correctly classified instances out of the total number of instances. It is defined as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (2.1)$$

Although accuracy is intuitive and easy to interpret, it may not be a reliable metric in the presence of class imbalance. In such situations, a model may achieve a high accuracy while still performing poorly on the minority class.

F1 Score

The F1 score is a metric that combines precision and recall into a single value, providing a balance between the two. It is particularly useful when both false positives and false negatives are of concern.

First, precision and recall are defined as follows:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (2.2)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (2.3)$$

The F1 score is then computed as the harmonic mean of precision and recall:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (2.4)$$

This metric is especially appropriate in scenarios with imbalanced datasets, where a trade-off between precision and recall must be considered.

Area Under the Curve (AUC)

The Area Under the Curve (AUC) typically refers to the area under the Receiver Operating Characteristic (ROC) curve. The ROC curve illustrates the relationship between the True Positive Rate (TPR) and the False Positive Rate (FPR) across different classification thresholds.

The TPR and FPR are defined as:

$$\text{TPR} = \frac{TP}{TP + FN} \quad (2.5)$$

$$\text{FPR} = \frac{FP}{FP + TN} \quad (2.6)$$

The AUC value can be interpreted as the probability that the model assigns a higher score to a randomly selected positive instance than to a randomly selected negative instance. An AUC value close to 1 indicates strong discriminative ability, whereas a value close to 0.5 suggests performance equivalent to random classification.

2.1.2 Studies done with the CBIS-DDSM Dataset

Next, we compare some of the work that has been done with the dataset, to analyze how they processed the data, which models they trained and which results were achieved.

- **Transfer Learning and Fine Tuning in Breast Mammogram Abnormalities Classification on CBIS-DDSM Database** [15]: the methodology proposed in this paper prepares the data by changing the resolution of the images and augmenting the contrast using the Contrast Limited Adaptive Histogram Equalization (CLAHE) [19] that enhances image quality by applying histogram equalization locally on small regions (tiles), which improves contrast and reveals fine details in different parts of the image. Next,

they find the region of interest (ROI) using a box surrounding the suspicious mass, and normalize that region again. Since these boxes can be of different shapes, they keep only the ROIs with an aspect ratio $r = \frac{w}{h}$ between 0.4 and 1.5. Lastly, the ROIs are resized to a final shape of 320×320 : they added a padding of 8 pixels and then cropped the center of the image, and filtered the image to reduce noise.

Next, a step of data augmentation is taken to prevent overfitting. This is achieved by applying rotations, shears, flips, zooms, etc. to the images. They considered the Train/Test split that the CBIS-DDSM dataset presents, but joined them to augment the data and then splitted them into different Train and Test sets once they had increased the size to 60 thousand images. They split it into 80% for training, 10% for validation and the last 10% for testing.

Lastly, they used the modified dataset in seven CNNs that were pretrained, and studied the performance of Transfer Learning. Since the problem approached in this paper is binary classification, they evaluated the results using a confusion matrices, Accuracy, F1 score and Area under the ROC curve. They saw best results in VGG16 [20], with so they fine-tuned it adjusting the initial model and obtained values of $AUC = 0.844$, $F_1 = 0.85$, $ACC = 0.844$.

- **Sensitivity-Based Breast Cancer Characterization on CBIS-DDSM Mammography Dataset Using Fine-Tuned EfficientNet-B2 Model** [21]: the preprocessing in this paper is specially different to the one we have seen above. The images are reduced to a constant resolution and no data augmentation processes were applied. Once more, they did not respect the Train-Test partition provided by the CBIS-DDSM dataset, but defined their own that was more balanced.

Since the amount of images corresponding to benign tumors is greater than the ones depicting malignant tumors, they opted for imbalance-conscious learning methods: an EfficientNet-B2, that was pretrained, followed by Global Average Pooling, a Fully Connected layer, Batch Normalization and a Sigmoid output layer. They fine-tuned the last 25 layers of the model and obtained $AUC = 0.679$, $ACC = 0.6496$, which is not bad considering that it had a small amount of trainable parameters.

- **Deep BI-RADS Network for Improved Cancer Detection from Mammograms** [22]: the uniqueness of this paper, compared with the two above, is that they do not use the full BIS-DDSM dataset, but instead they take a subset and enrich it with additional clinical data: it utilizes the textual metadata associated with each mammogram, that provides extra information about the kind of lesion and breast descriptors of the Breast Imaging Reporting and Data System (BI-RADS) [23].

The architecture consists of two branches, each composed of 6 stacked identical attention-based layers, that takes images and textual descriptors. They process the images by scaling them to 1024×1024 pixels, and conduct data augmentation by doing flips and elastic deformation. Then, features are extracted from the images using BiT layers, that are pre-trained. That is followed by various attention layers, that range from Cross-Attention, Self-Attention, etc. The data is fed to them in mini-batches of 16. The best results obtained in this paper, that tried various (Query, Key, Value) configurations is $AUC = 0.878$, $F_1 = 0.780$, $ACC = 0.796$.

The following table summarizes the lines above.

Table 2.1: Summary of CBIS-DDSM binary classification studies

Study	AUC	ACC	F_1
Transfer Learning and Fine Tuning in Breast Mammogram Abnormalities Classification on CBIS-DDSM Database [15]	0.844	0.844	0.85
Sensitivity-Based Breast Cancer Characterization on CBIS-DDSM Mammography Dataset Using Fine-Tuned EfficientNet-B2 Model [21]:	0.679	0.6469	-
Deep BI-RADS Network for Improved Cancer Detection from Mammograms [22]	0.878	0.796	0.780

From all these papers we can learn that there is not one standardized way to treat the volume of data in the CBIS-DDSM dataset, nor the imbalance that it presents. All of them use pre-trained datasets and fine-tune them in the last step in order to achieve the best results, which is aligned with the plan layed out in this project. It is notable how the projects that augmented the data and added other forms of data such as text obtained the best results, while the project that kept the dataset intact presented worse AUC and ACC.

2.2 Visual Transformers (ViT)

This section introduces Vision Transformers (ViT) and explains how they operate. They are also compared with traditional Convolutional Neural Networks (CNNs), which are the more traditional alternative to address image classification problems like the one proposed in this project.

Vision Transformers were introduced in 2021 in the paper *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale* [24]. The paper studies how Transformer architectures, which are very successful in Natural Language Processing, can be adapted to Computer Vision tasks.

The idea proposed by Dosovitskiy et. al. [24] is to remove convolution completely and instead apply the transformers directly to images, that are codified as sequences of tokens. This new approach reduces the resources needed, whilst maintaining excellent results when pre-trained with large amounts of data. Consequently, we can split Vision Transformers into two stages:

1. **Image to patch embeddings:** to create a one-dimensional sequence of token embeddings to feed the transformer from a 2D image, ViT first converts an image into a sequence of patches. Given an input image $x \in \mathbb{R}^{H \times W \times C}$, where (H, W) is the resolution of the image and C is the number of channels (3 in case of an RGB image), the image is divided into $N = \frac{HW}{P^2}$ 2D non-overlapping patches of size $P \times P$: $X_p \in \mathbb{R}^{N \times P^2 \times C}$. N will be the input sequence length for the Transformer.

Each of the transformer layers uses a constant latent vector of size D : the N image patches need to be flattened into D dimensions using a trainable linear projection, that results into the *patch embedding*, similar to word embeddings in NLP.

$$z_0 = [x_{\text{class}}; x_p^1 E; x_p^2 E; \dots; x_p^N E] + E_{\text{pos}}$$

where $E \in \mathbb{R}^{(P^2 \cdot C) \times D}$, $E_{\text{pos}} \in \mathbb{R}^{(N+1) \times D}$ and x_{class} is a learnable classification token.

Position embeddings are added to the patch embeddings to retain positional information. It is done using standard learnable 1D position embeddings. The resulting sequence of embedding vectors serves as input to the encoder.

2. **Transformer encoder:** after embedding, the sequence of patches is passed to a standard Transformer encoder. The encoder consists of multiple identical layers, each containing:
 - Multi-Head Self-Attention (MSA): introduced in *Attention Is All You Need* [25], it is a mechanism to compute attention between all input tokens using multiple parallel attention heads to capture diverse relationships.
 - Feed-Forward Network (MLP): consisting of fully connected layers with GELU activation functions[26]: it consists in non-linear Gaussian activation.
 - Layer Normalization (LN) before every block.
 - Residual connections after every block.

The work done in *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale* [24] found that the Vision Transformers are, in general, less robust to small perturbations than conventional CNNs: they have less inductive bias. This is because the position embeddings at the initialization time carry no information about the positions of the patches. Since they have to be learned from scratch, and only MLP layers are invariant to translations. Dividing the image into patches breaks the two-dimensional neighborhood structure but, when trained with large datasets, having learned these spatial relationships from the data makes it more flexible.

For these reasons, the typical use case of a ViT requires pre-training with a big dataset and then fine-tuning to the smaller task (breast cancer image classification). The initial study [24] found that pre-trained ViTs dominate CNNs on the performance/compute trade-off.

This original proposal for Visual Transformers [24] compared them to Residual Networks (ResNets), those are a particular type of CNNs introduced in 2015 by Kaiming He et. al., in a paper titled *Deep Residual Learning for Image Recognition* [27]. ResNets are designed to enable the effective training of very deep models by introducing *residual learning*. Instead of directly learning a desired mapping $H(x)$, each block learns a residual function $F(x) = H(x) - x$, which is combined with the input through identity shortcut (skip) connections. The results show that gradients flow more easily through the network; and it mitigates the degradation problem where deeper networks perform worse than shallower ones.

As it is a type of Convolutional Neural Network, it processes images by using convolutional layers with small kernels (e.g., 3×3). The main trait of them, as we just mentioned, is that the input is added back in every convolutional block. On the other hand, Visual Transformers do not use convolution whatsoever, and trade off that local context for global context. The following figure 2.1 illustrates these differences.

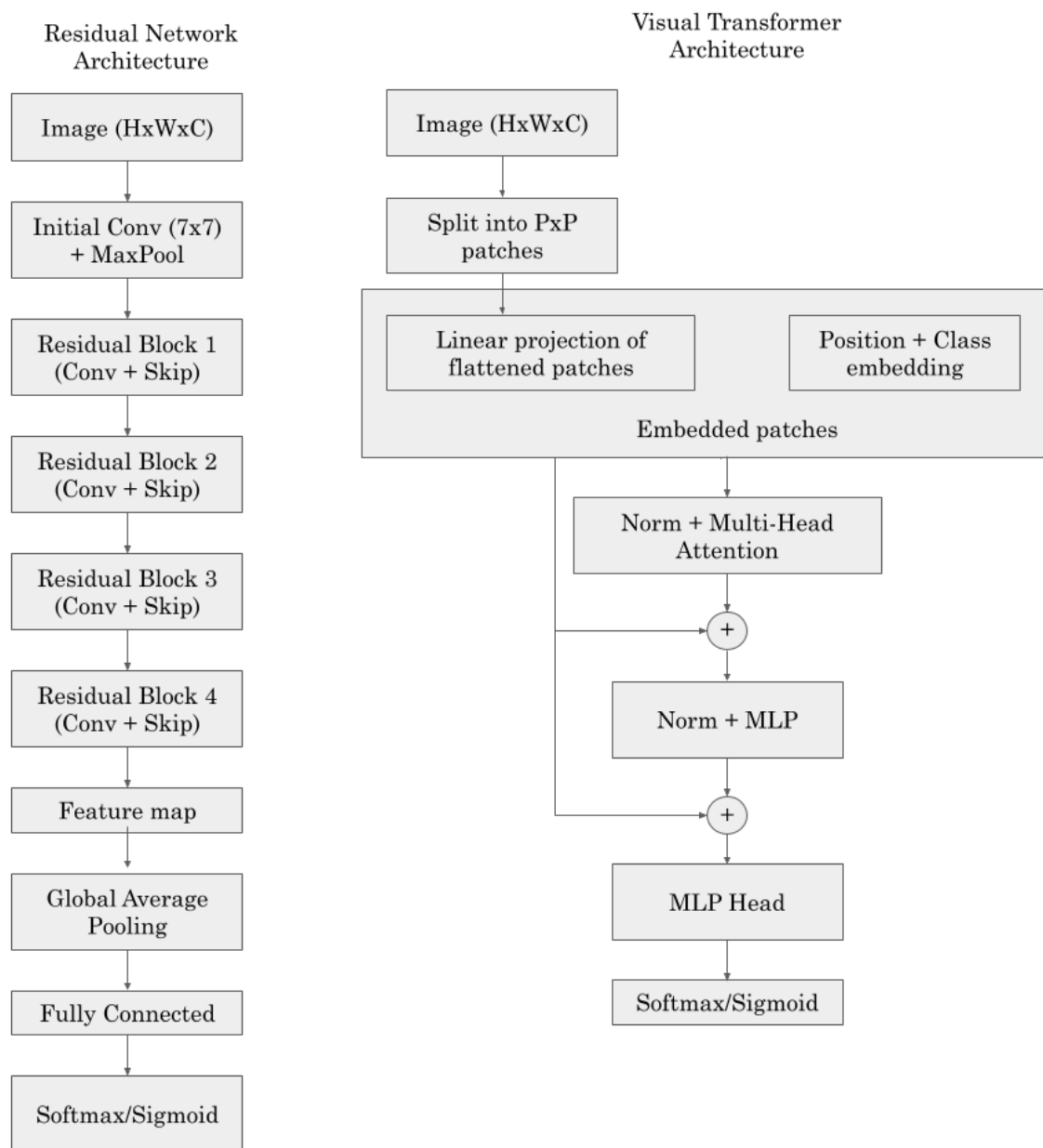


Figure 2.1: Residual Network and Visual Transformer Architectures

2.3 Topological Data Analysis (TDA) applied to images

This section explains what topological descriptors represent, and how to compute homology descriptors that can be embedded in a Visual Transformer: persistence images.

Topology is a branch of mathematics that studies the shape of objects and their main properties that do not change under continuous transformations. In the context of topological data analysis (TDA), these concepts can be applied to study the structure of grayscale images, where the pixel intensity values define the data representation. One of the most commonly used techniques in TDA is persistent homology, which allows to track the appearance and disappearance of topological features, such as connected components and holes, across different intensity thresholds.[28]

To obtain topological descriptors from a grayscale image, we consider each pixel to be associated with an intensity value. This image can be interpreted as a function defined on a grid, in which the intensity values represent different levels of the structure. To analyze this image using TDA, we study how the structure changes when we progressively vary a threshold over the intensity values.

For each threshold value, we construct a binary image by selecting the pixels that satisfy the given condition. In this way, we can observe how many connected components and holes are present at each stage. As the threshold changes, new components can appear, existing ones can merge, and holes can be created or filled. Persistent homology allows to track these changes by recording the birth and death of each topological feature.

These features are represented using persistence diagrams (PDs), which describe the lifetime of each topological feature from its birth to its death: each point in the diagram with coordinates (b_i, d_i) corresponds to a feature with birth b_i and death d_i . These diagrams provide a compact summary of the topological structure of the image across all thresholds. Since PDs are not directly suitable for most machine learning models, they are usually transformed into a finite-dimensional representations. This project uses the following:

1. **Betti curves:** show how many descriptors are alive at any given time. They can be compute for the topological descriptors of each homology class: in grayscale images: H_0 and H_1 . These curves can be discretized and fed as vectors to classifier models.[28]
2. **Landscapes:** given a persistence diagram, the landscapes can be computed by rotating it and laying it against the diagonal. Then, tent functions are computed for every point above the diagonal: lines paralel to the original axis that connect the point with the diagonal. The landscape functions can be discretized and fed as vectors to classifier models.[28]
3. **Silhouettes:** are a weighted average of tent functions with weights $w_i > 0$. A popular choice is $w_i = (d_i - b_i)^p$ where (b_i, d_i) is each point of the persistence diagram and p is a positive integer. Again, these functions can be discretized and fed as vectors to classifier models.[28]
4. **Persistence images (PIs):** the persistence diagram is converted into a persistence image by mapping each point into a continuous domain (usually transposing the persistence diagram) and applying a suitable probability distribution and weighting func-

tion, $f : \mathbb{R}^2 \rightarrow \mathbb{R}$, that must be zero along the horizontal axis, continuous and piecewise differentiable[29]. The example in [29, Image 1] illustrates this procedure.

The following diagram depicts this process with an example grayscale image. 2.2

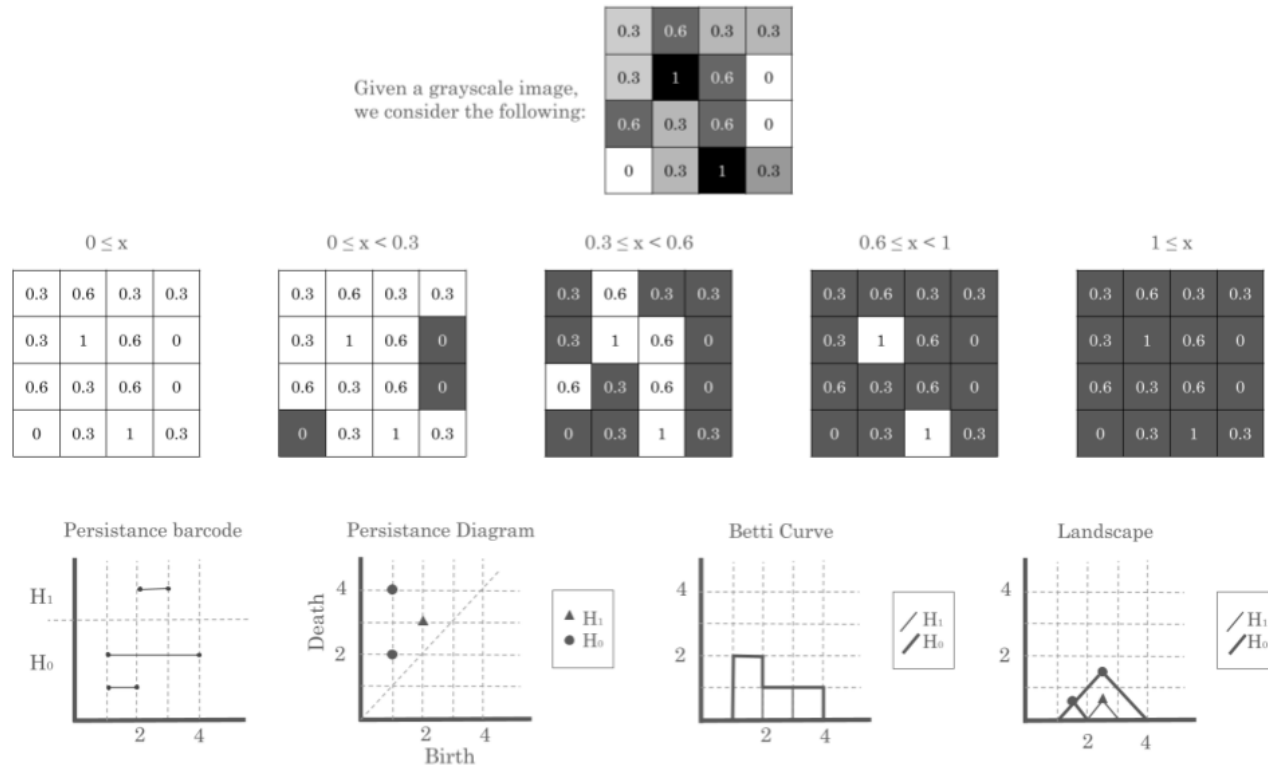


Figure 2.2: How to compute a persistence diagram: given a grayscale image, we consider how many connected components (H_0) and cycles (H_1) there are. By monitoring when they appear (birth) and when they get absorbed by another component, we can draw a Barcode Diagram, that has one parallel line for each component, and a Persistence Diagram, that has one point in coordinates (B, D) corresponding to each birth (B) and Death (D) of a component.

2.4 Combining ViT and TDA:

Finally, this section explores existing systems that use Topological Data Analysis to improve a Vision Transformer, how it is embedded into the architecture, what applications were explored, and the results achieved.

Recent works have shown that, while Vision Transformers are powerful in capturing global dependencies through self-attention, they may lack robustness in representing intrinsic geometric and topological structures of the data. The integration of TDA can help address this limitation, providing complementary information about the shape and connectivity of data distributions.

The work in *TDA-ViT: A Transformer-Based Framework for Unified Urdu Text Recognition via Topological and Visual Feature Fusion* [30] introduces a Transformer-based framework that combines visual and topological features for the task of Urdu text recognition. The approach followed consists of four stages:

1. Topological descriptors are extracted from images of Urdu text, and stored into persistence images 2.3.
2. Raw and persistent images are encoded using a Vision Transformer for feature embedding. This produces a high-dimensional feature sequence that encodes both pixel-level and topological information.
3. The resulting sequences are passed to a T5-based encoder (Transformer used for Natural Language Processing).
4. Lastly, the sequences are decoded using a GPT-2 based decoder.

This hybrid approach improves robustness to noise and distortions, which are common in handwritten or low-quality text images. Experimentally, the model demonstrates improved recognition accuracy compared to standard ViT-based baselines, highlighting the benefit of incorporating topological descriptors.

Despite these promising contributions, there are still several limitations and open challenges that motivate further research. First, the application domains explored in the current literature remain relatively limited, with a strong focus on text recognition. There is a lack of investigation in other domains such as medical imaging, where topological structures are often highly informative. In particular, mammography datasets such as CBIS-DDSM contain subtle structural patterns (e.g., mass irregularity, spiculation, and boundary complexity) that are not explicitly addressed in the existing works.

Second, other topological descriptors coming from images can be combined with visual transformers via vector composition, and that has not been broadly addressed in the papers found.

Lastly, the computational cost and scalability of TDA components are not deeply analyzed in these works. Persistent homology can be expensive to compute, especially for high-resolution medical images, and its impact on training efficiency remains an open question.

There is a gap in the literature regarding the design of a TDA-enhanced Vision Transformer architectures for medical image classification. In this project, we aim to address it by investigating how topological features can be integrated into a ViT-based pipeline for binary classification of mammography images from the CBIS-DDSM dataset into benign and malignant categories.

Chapter 3

Materials and methods

This chapter describes the design and implementation of the experiments developed to evaluate the two hypotheses stated in Section 1.1. It covers the data preparation process, the image preprocessing techniques applied, the extraction of topological descriptors, the model architectures designed, and the experimental protocol followed. All experiments were implemented in Python using PyTorch [31] as the deep learning framework.

3.1 Data preparation

This section covers in detail how all data is obtained and prepared before starting the preprocessing.

3.1.1 Dataset selection and scope

As introduced in Section 2.1, this project uses the CBIS-DDSM dataset [6] for binary classification of mammographic lesions into benign and malignant categories. The dataset provides both breast masses and breast calcifications; however, this work focuses exclusively on the **breast mass** subset to maintain a controlled experimental scope and reduce computational cost.

The CBIS-DDSM dataset provides, for each lesion, three types of images: the full mammogram, a pre-cropped region of interest (ROI), and a binary ROI mask. The pathology labels include **MALIGNANT**, **BENIGN**, and **BENIGN_WITHOUT_CALLBACK**. Following the simplification adopted in the literature [15], the latter two categories are merged into a single **BENIGN** class, resulting in a binary classification task.

3.1.2 Data loading and file resolution

The CBIS-DDSM images are stored in DICOM format. Each DICOM file is loaded using the `pydicom` library [32], and the pixel array is normalized to the $[0, 255]$ range as an unsigned 8-bit integer:

$$I_{\text{norm}} = \left\lfloor \frac{I - I_{\min}}{I_{\max} - I_{\min}} \cdot 255 \right\rfloor \quad (3.1)$$

During the exploratory analysis, two issues were identified in the dataset file structure:

1. The file names provided in the CSV metadata do not match the actual file names on disk (the CSV references 000000.dcm and 000001.dcm, but the actual files have UUID-based names).
2. The ROI mask and the cropped image often share the same folder, making them indistinguishable by path alone.

To resolve these issues, the implementation identifies files by their **size**: within a shared folder, the smaller file corresponds to the cropped mammogram (fewer pixels), while the larger file corresponds to the ROI mask (which covers the full mammogram dimensions). This heuristic was validated manually and proved reliable across the entire dataset.

3.1.3 Train, validation, and test splits

The CBIS-DDSM dataset provides predefined training and testing partitions split by patient, ensuring that no patient appears in both sets. This project respects these original partitions to maintain comparability with the literature.

From the training partition, a stratified 85/15 split is performed to create a validation subset, using `StratifiedShuffleSplit` from `scikit-learn` [33] with a fixed random seed of 2 for reproducibility. The stratification ensures that the class distribution (benign vs. malignant) is preserved in both the training and validation subsets.

The resulting approximate sizes for the breast mass subset are:

- Training set: ~587 samples (85% of the original training partition).
- Validation set: ~104 samples (15% of the original training partition).
- Test set: ~201 samples (the original test partition).

3.1.4 Image caching

To avoid repeated DICOM parsing and file resolution during training, all cropped mammogram images are precomputed and stored as NumPy `.npy` arrays. A manifest CSV file (`cached.csv`) records the path and label for each sample, enabling efficient data loading during experimentation.

3.2 Image preprocessing

The preprocessing pipeline is designed to enhance the features in mammographic images and to reduce noise. It consists of two sequential operations applied to the grayscale image, and follows what was done in the other papers explored:

1. **Contrast Limited Adaptive Histogram Equalization (CLAHE)** [19]: is applied to enhance local contrast. Unlike standard histogram equalization, CLAHE operates on small tiles of the image and limits the amplification of contrast to avoid noise enhancement. The parameters used are: Clip limit: 2.0 and Tile grid size: 8×8 . These values are standard in the medical imaging literature and were adopted from previous work on the CBIS-DDSM dataset [15].

2. **Gaussian denoising:** After contrast enhancement, a Gaussian blur with a 5×5 kernel is applied to reduce high-frequency noise introduced by the CLAHE step. The standard deviation is computed automatically by OpenCV [34] based on the kernel size.

The complete preprocessing function applies CLAHE first, followed by Gaussian denoising. Figure 3.1 illustrates the effect of this pipeline on a sample mammogram.

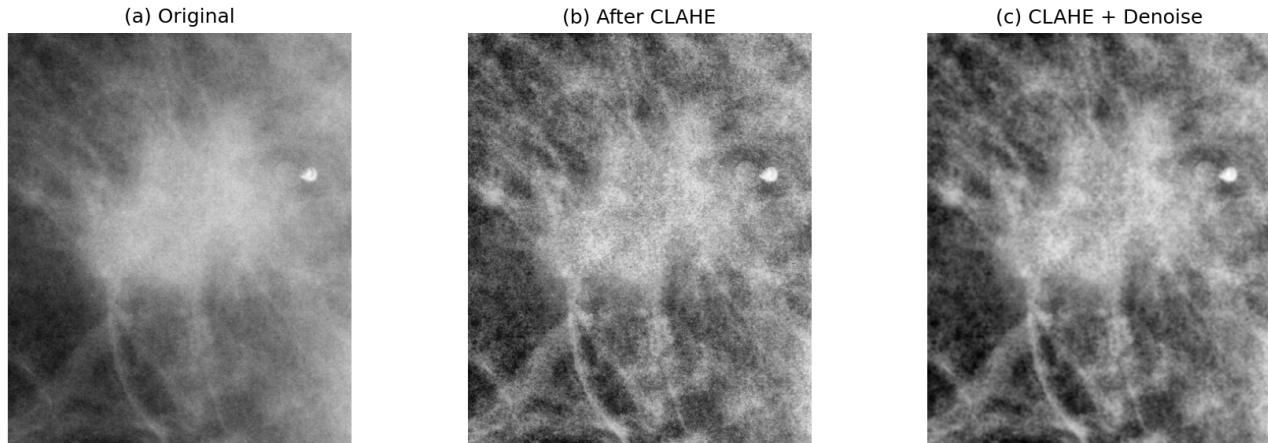


Figure 3.1: Effect of the preprocessing pipeline on a sample cropped mammogram: (a) original image, (b) after CLAHE, (c) after CLAHE + Gaussian denoise.

3.3 Topological feature extraction

This section describes how topological descriptors are extracted from mammographic images using the persistent homology framework introduced in Section 2.3. Two types of topological representations are computed: persistence images (for the dual-ViT architecture) and compact vector descriptors (for the linear fusion architecture). Details on those architectures are found in the next section 3.4.

Topological features are computed from two different image sources to evaluate which provides more informative structural information:

1. **Pre-cropped lesion:** the rectangular ROI crop provided by the dataset, which contains the lesion and some surrounding tissue.
2. **Masked mammogram crop:** the full mammogram multiplied by the binary ROI mask, then cropped to the bounding box of the mask with 15% padding. This isolates the lesion boundary more precisely.

3.3.1 Persistence images

Persistence images are computed using the `giotto-tda` library [35]. The process follows these steps:

1. **Cubical persistence:** the grayscale image is interpreted as a cubical complex, where pixel intensities define the filtration values. The `CubicalPersistence` class computes persistent homology for dimensions 0 (connected components) and 1 (cycles/holes).
2. **Persistence image transformation:** the resulting persistence diagram is transformed into a fixed-size persistence image of resolution 100×100 using the `PersistenceImage` class, which applies a Gaussian kernel and a linear weighting function as described in [29].

The output is a tensor of shape $(2, 100, 100)$: one 100×100 image for H_0 features and one for H_1 features.

3.3.2 Compact vector descriptors

Compact vector descriptors are computed using the `gudhi` library [36][37]. For each image, a cubical complex is constructed and persistent homology is computed for dimensions 0 and 1. Infinite persistence pairs (features that never die) are excluded. Three types of vectorizations are then computed at a resolution of 100 sample points:

- **Betti curves** [28]: discretized functions counting the number of features alive at each filtration value. One curve per homology dimension: $2 \times 100 = 200$ values.
- **Persistence landscapes** [28]: five landscape functions per homology dimension, each discretized at 100 points: $2 \times 5 \times 100 = 1000$ values.
- **Persistence silhouettes** [28]: weighted averages of tent functions, one per homology dimension: $2 \times 100 = 200$ values.

The three vectorizations are concatenated into a single descriptor of dimension $200 + 1000 + 200 = 1400$.

Since TDA computation is expensive (cubical persistence on high-resolution images), all topological descriptors are precomputed and cached as `.npy` files before training begins. The manifest CSV is extended with columns pointing to the corresponding TDA file for each sample.

3.4 Model architectures

Three model architectures are used across the seven experiments. All are based on the Vision Transformer (ViT-B/16) [24] pretrained on ImageNet, loaded via the `timm` library [38].

3.4.1 ViT classifier

The baseline architecture uses the standard ViT-B/16 model with its classification head replaced by a single linear layer that outputs one logit for binary classification:

$$\hat{y} = W \cdot z_{\text{CLS}} + b \quad (3.2)$$

where $z_{\text{CLS}} \in \mathbb{R}^{768}$ is the CLS token output from the transformer encoder, and $W \in \mathbb{R}^{1 \times 768}$, $b \in \mathbb{R}$ are the learnable parameters of the classification head.

Input images are resized to 224×224 pixels and replicated across three channels (since the original images are grayscale) to match the pretrained model's expected input format.

Figure 3.2 illustrates this architecture.

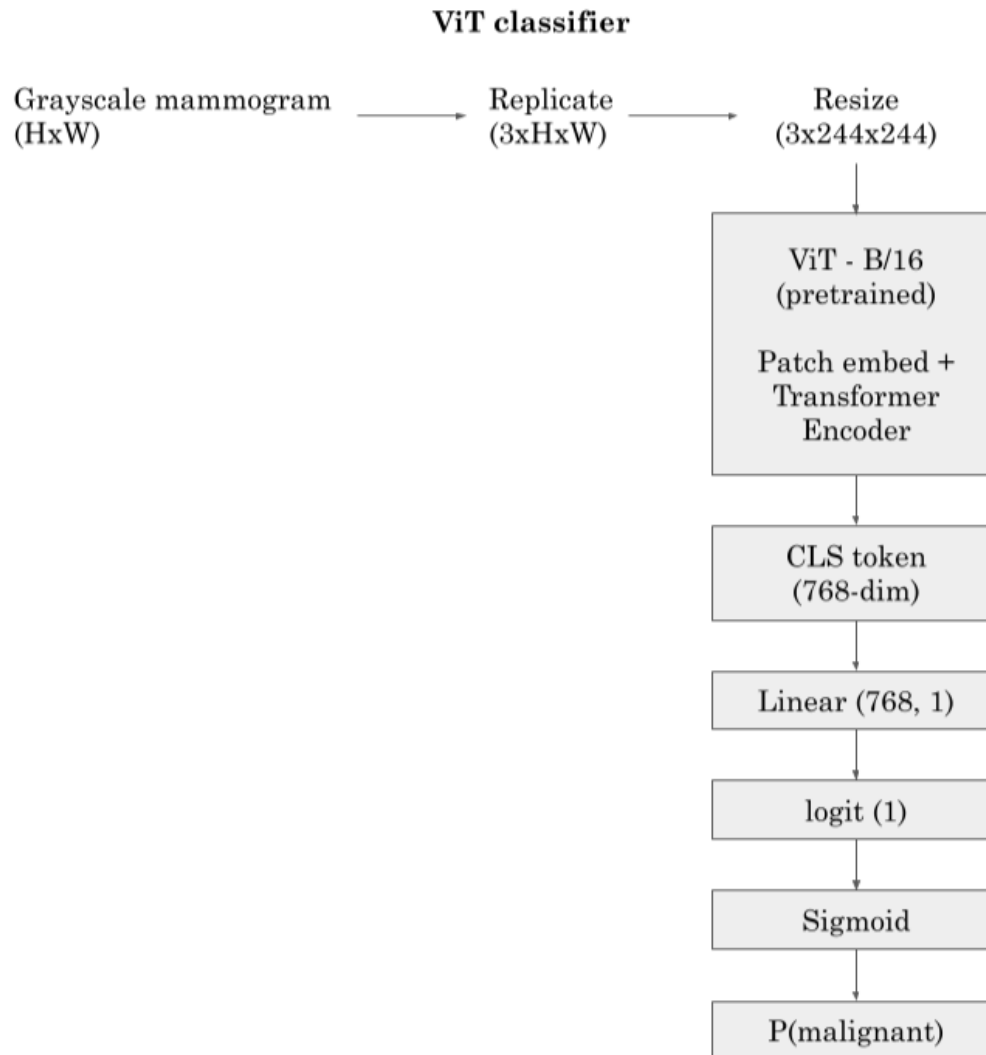


Figure 3.2: ViT classifier architecture: the grayscale mammogram is replicated to 3 channels, resized to 224×224 , and passed through the pretrained ViT-B/16. The CLS token is projected to a single logit via a linear layer.

3.4.2 Dual-ViT fusion model

This architecture processes the mammogram and its persistence image through two separate ViT-B/16 branches. The persistence image (shape $2 \times 100 \times 100$) is padded with a zero channel to form a 3-channel input and resized to 224×224 via bilinear interpolation. The CLS tokens from both branches are concatenated and passed through a classifier head:

$$\hat{y} = \text{MLP}([z_{\text{CLS}}^{\text{img}} \| z_{\text{CLS}}^{\text{TDA}}]) \quad (3.3)$$

where $\|$ denotes concatenation, and the MLP consists of:

$$\text{Linear}(1536, 256) \rightarrow \text{ReLU} \rightarrow \text{Linear}(256, 1)$$

Figure 3.3 illustrates this architecture.

3.4.3 Linear fusion model

This architecture combines the ViT image branch with a linear projection of the compact TDA vector descriptor. The ViT branch extracts the 768-dimensional CLS token from the mammogram. The TDA branch projects the 1400-dimensional vector through a linear layer to 128 dimensions. Both representations are concatenated and classified:

$$\hat{y} = \text{MLP}([z_{\text{CLS}}^{\text{img}} \| W_{\text{TDA}} \cdot v_{\text{TDA}}]) \quad (3.4)$$

where $v_{\text{TDA}} \in \mathbb{R}^{1400}$ is the compact descriptor, $W_{\text{TDA}} \in \mathbb{R}^{128 \times 1400}$ is the linear projection, and the MLP is:

$$\text{Linear}(768 + 128, 256) \rightarrow \text{ReLU} \rightarrow \text{Linear}(256, 1)$$

Figure 3.4 illustrates this architecture.

3.5 Data augmentation

Two levels of data augmentation are implemented, using transformations that are appropriate for mammographic images, which have no inherent orientation preference once cropped to the ROI:

1. **Basic augmentation:** Basic augmentation applies random geometric transformations with 50% probability each: random horizontal flip and random vertical flip.
2. **Aggressive augmentation:** In addition to the basic flips, aggressive augmentation applies: random rotation: $\pm 15^\circ$, with border reflection, random zoom/crop: scale factor uniformly sampled from $[0.85, 1.0]$, followed by resize to original dimensions and random shear: horizontal shear factor uniformly sampled from $[-0.1, 0.1]$, applied with 50% probability. This configuration is inspired by the augmentation strategy described in [15], adapted to operate without expanding the dataset size: augmentations are applied during training rather than generating new samples offline.

3.6 Training protocol

All experiments follow the same training protocol to ensure fair comparison:

Dual ViT classifier

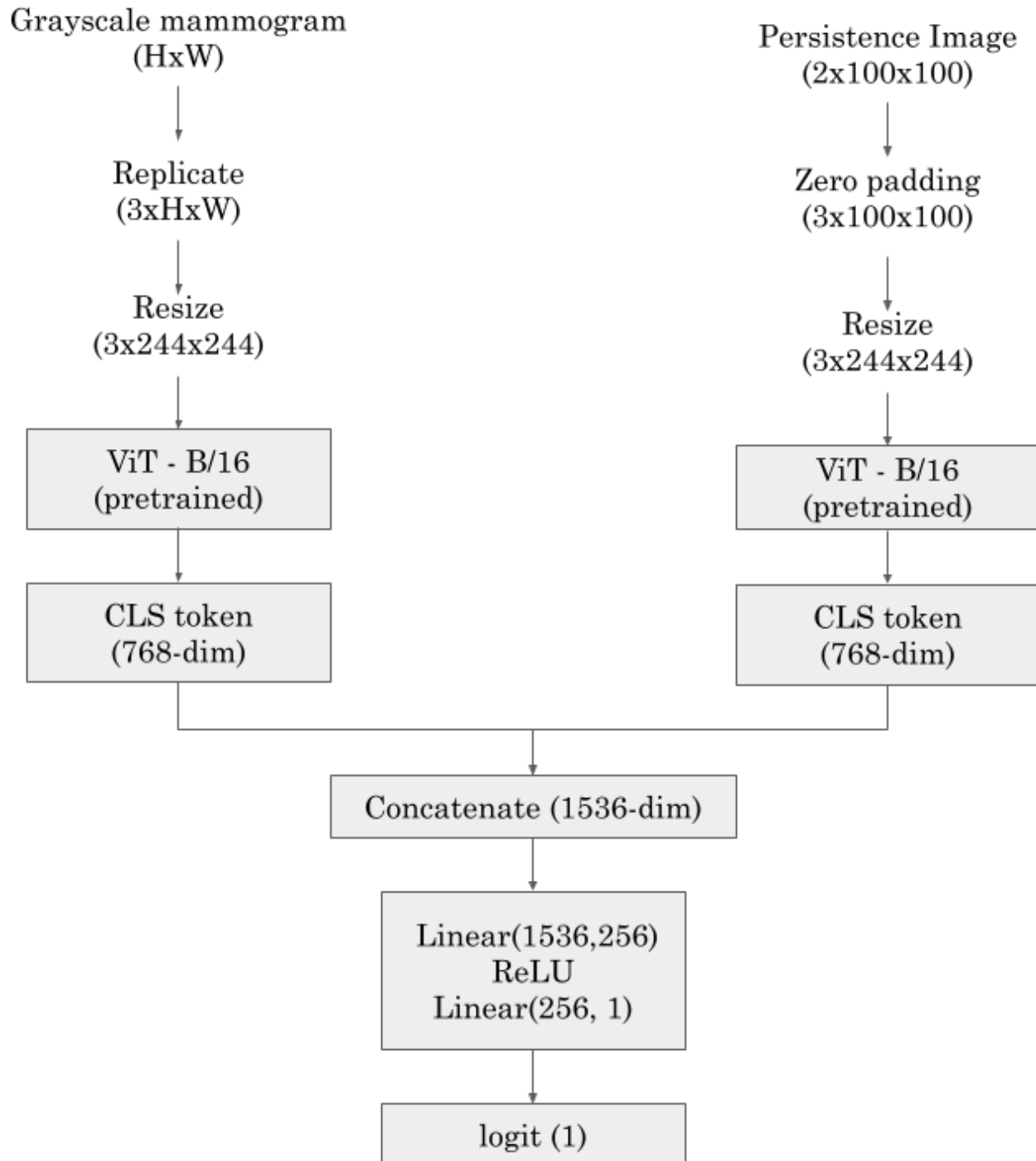


Figure 3.3: Dual-ViT fusion architecture: the mammogram and the persistence image are each processed by a separate pretrained ViT-B/16. The two CLS token embeddings (768-dim each) are concatenated and classified by an MLP.

ViT and TDA vector classifier

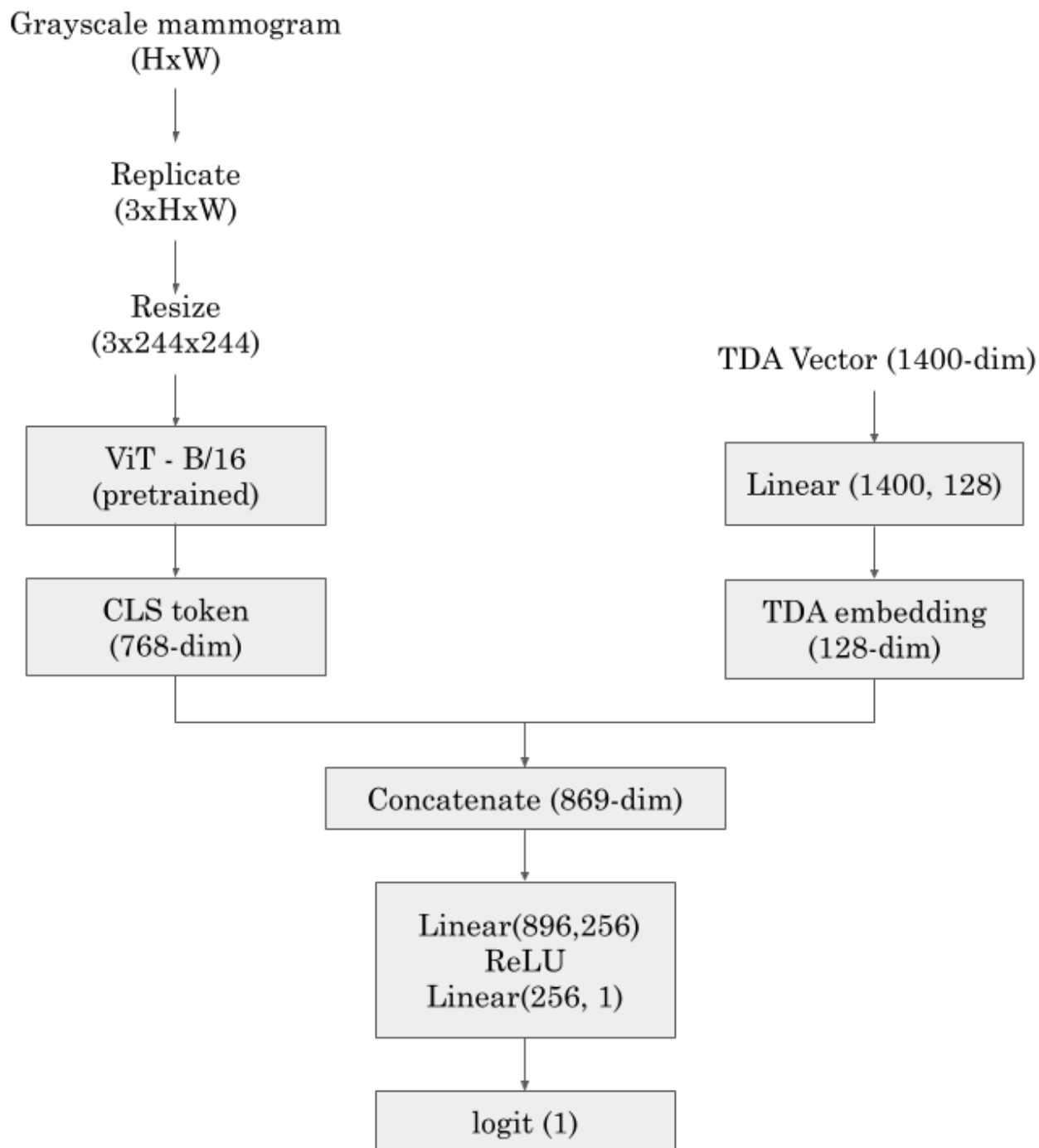


Figure 3.4: Linear fusion architecture: the mammogram is processed by ViT-B/16 (768-dim CLS token), while the 1400-dim TDA vector is projected to 128 dimensions. Both are concatenated and classified by an MLP.

1. **Loss function:** Binary Cross-Entropy with Logits Loss (`BCEWithLogitsLoss`) is used, with a `pos_weight` parameter to handle class imbalance. The weight is computed as the ratio of negative to positive samples in the training set:

$$w_{\text{pos}} = \frac{N_{\text{neg}}}{N_{\text{pos}}} \quad (3.5)$$

2. **Optimizer and learning rate schedule:** the Adam optimizer [39] is used with a learning rate of 10^{-4} . A Cosine Annealing learning rate scheduler [40] is applied over the total number of epochs ($T_{\text{max}} = 30$), gradually reducing the learning rate to zero following a cosine curve.
3. **Early stopping:** to prevent overfitting, early stopping is implemented with a patience of 5 epochs. Training halts if the validation loss does not improve for 5 consecutive epochs. The model checkpoint with the lowest validation loss is retained for final evaluation.
4. **Evaluation metrics:** models are evaluated on the held-out test set using the metrics defined in Section 2.1: Accuracy, F1 Score, and Area Under the ROC Curve (AUC). The AUC is used as the primary metric for comparison, as it is threshold-independent and robust to class imbalance.
5. **Hyperparameters:** table 3.1 summarizes the training hyperparameters, which are shared across all experiments.

Table 3.1: Training hyperparameters

Parameter	Value
Batch size	16
Learning rate	10^{-4}
Optimizer	Adam
Scheduler	Cosine Annealing ($T_{\text{max}} = 30$)
Maximum epochs	30
Early stopping patience	5 epochs
Random seed	2
Classification threshold	0.5

3.7 Experimental design

Seven experiments are designed to systematically evaluate the two hypotheses. They are organized in two groups:

3.7.1 Hypothesis 1: Impact of preprocessing (E1–E3)

- **E1: Baseline ViT:** No preprocessing, basic augmentation (flips only). Uses the ViT classifier (Section 3.4.1). This establishes the baseline performance of a pretrained ViT on raw mammographic crops.
- **E2: ViT with preprocessing:** CLAHE + Gaussian denoise applied, basic augmentation. Same architecture as E1. This isolates the effect of preprocessing.
- **E3: ViT with preprocessing + aggressive augmentation:** Same as E2 but with aggressive augmentation (rotation, zoom, shear). This evaluates whether stronger augmentation can compensate for the small dataset size.

3.7.2 Hypothesis 2: Impact of TDA features (E4–E7)

All TDA experiments use the best configuration from H1 (preprocessing enabled, basic augmentation) as their baseline, and add topological information through different fusion strategies:

- **E4: Dual-ViT with persistence images (cropped lesion):** Uses the DualViTFusionModel (Section 3.4.2). The persistence image is computed from the pre-cropped lesion ROI.
- **E5: Dual-ViT with persistence images (masked mammogram):** Same architecture as E4, but the persistence image is computed from the masked mammogram crop (full mammogram $\times$ ROI mask).
- **E6: Linear fusion with TDA vectors (cropped lesion):** Uses the FusionModel (Section 3.4.3). The 1400-dimensional compact descriptor is computed from the pre-cropped lesion ROI.
- **E7: Linear fusion with TDA vectors (masked mammogram):** Same architecture as E6, but the compact descriptor is computed from the masked mammogram crop.

Table 3.2 summarizes the experimental configurations.

Table 3.2: Summary of experimental configurations

Experiment	Preprocessing	Augmentation	TDA input	Architecture
E1	No	Basic	—	ViT classifier
E2	Yes	Basic	—	ViT classifier
E3	Yes	Aggressive	—	ViT classifier
E4	Yes	Basic	PI (crop)	Dual-ViT
E5	Yes	Basic	PI (masked)	Dual-ViT
E6	Yes	Basic	Vector (crop)	Linear fusion
E7	Yes	Basic	Vector (masked)	Linear fusion

3.8 Implementation and reproducibility

The project is implemented as a modular Python codebase with the following structure:

```
src/
  datasets/
    experiment_dataset.py    # PyTorch Dataset class
  models/
    vit_model.py            # ViT classifier
    fusion_model.py         # DualViT and Linear fusion
  tda/
    tda_features.py         # TDA extraction functions
  training/
    experiment_runner.py    # Training loop and evaluation
  utils/
    image_utils.py          # DICOM loading and path resolution
    preprocess.py           # CLAHE + denoise pipeline
```

All experiments are orchestrated from a Jupyter notebook (`notebook.ipynb`) that calls the modular components. Results (metrics, learning curves, and predictions) are persisted to JSON files in a `checkpoints/` directory, enabling survival across kernel restarts and reproducibility of the analysis.

Looking at it file by file, the structure is:

- **notebook.ipynb**: notebook that describes the project and triggers the experiment execution.
- **src/utils/**:
 - **image_utils.py**: has seven functions to take care of image loading and path resolution:
 - * `load_dicom(path)`: given a full path, it uses `pydicom` [32] to load the image and transform it into a `numpy` matrix [41]. It normalizes the image to have values between 0 and 255.
 - * `get_all_dcm_files_with_size(rel_path)`: given a relative path, it builds the full path using the `BASE_PATH` defined in the same `image_utils.py` file, whilst taking into account the long path limitations in Windows OS. With the `os` module it extracts all files with `.dcm` extension in said path, and sorts them by size (as described above, it will be used to distinguish masks from cropped mammograms).
 - * `resolve_mammogram_path(rel_path)`: extracts the mammogram path from a relative path, taking into account that the file name might be wrong. If more than one files are found in the provided folder, it raises an exception.
 - * `resolve_cropped_mammogram_path(rel_path)`: extracts the cropped mammogram path from a relative path, taking into account that the file name might be wrong. If one file is found in the path, it returns the path. If two files are

found, it returns the path to the smallest one. If more than two files are found in the provided folder, it raises an exception.

- * `resolve_roi_mask_path(rel_path)`: extracts the ROI mask path from a relative path, taking into account that the file name might be wrong. If one file is found in the path, it returns the path. If two files are found, it returns the path to the biggest one. If more than two files are found in the provided folder, it raises an exception.
- * `cache_cropped_mammogram_images_as_np_arrays(save_dir, df)`: given a dataset with "cropped image file path" and "pathology" columns, it (1) resolves all the paths to the cropped mammogram images (2) converts them into numpy arrays and saves them in the `save_dir` provided and (3) saves a `.csv` file in that same folder with two columns: path to the converted image and the corresponding label, that is 1 if it is malignant and 0 if it is either benign or Benign without callback. This function is used to reduce the time loading `dcm` files for every execution.
- * `extract_roi_crop(mammogram, mask, padding_ratio=0.15)`: extracts the ROI crop from a mammogram and applies the mask to it.
- **preprocess.py**: contains three functions for image preprocessing:
 - * `apply_clahe(image)`: applies Contrast Limited Adaptive Histogram Equalization (CLAHE) [19] to a grayscale image using `cv2.createCLAHE` [34] with `clipLimit=2.0` and `tileGridSize=(8,8)` to enhance local contrast.
 - * `denoise(image)`: applies a Gaussian blur with a 5×5 kernel using `cv2.GaussianBlur` [34] to reduce noise.
 - * `preprocess(image)`: chains `apply_clahe` followed by `denoise`, returning the pre-processed image.

- **src/models/**:

- **vit_model.py**: defines the `ViTClassifier` class, a PyTorch `nn.Module` [31] that wraps a pretrained Vision Transformer. It uses `timm.create_model` [38] to load `vit_base_patch16_224` [24] (ViT-B/16, pretrained on ImageNet). The classification head is replaced with a single `nn.Linear` [31] layer that maps from the ViT's hidden dimension (768) to 1 output (binary classification logit). The forward method passes the input through the full ViT and returns the raw logit (during training, `BCEWithLogitsLoss` [31] applies sigmoid internally) 3.2.
- **fusion_model.py**: defines two fusion architectures for combining ViT image features with TDA topological features:
 - * **FusionModel**: combines a Visual Transformer to process the images with a Linear Layer to process the vectorial topological descriptors. It loads the same `vit_base_patch16_224` backbone but replaces the head with `nn.Identity` [31] to extract the 768-dimensional CLS token. A separate TDA branch (`tda_fc`, an `nn.Linear` layer) projects the flattened TDA vector to 128 dimensions. The forward method concatenates both feature vectors using `torch.cat` [31] and passes the result through a classifier head: 3.4. The constructor accepts a

`tda_input_dim` parameter (default 1400) to match the vector descriptor dimensionality.

- * **DualViTFusionModel**: combines a Visual Transformer to process the images with another Visual Transformer to process the Persistence Images (PI). Both of them are separate `vit_base_patch16_224` branches. Both heads are replaced with `nn.Identity` to extract 768-dim CLS tokens. The forward method adapts the PI input (shape batch, 2, 100, 100 for H0 and H1 homology dimensions) to 3-channel ViT input by padding with a zero channel and resizing to 224×224 using `[F.interpolate]` (<https://pytorch.org/docs/stable/generated/torch.nn.functional.interpolate.html> find more details on the 3.7 section below. The two CLS tokens are concatenated via `torch.cat` and passed through a classifier: 3.3.

- **src/datasets/experiment_dataset.py**: defines the `ExperimentDataset` class, a PyTorch Dataset that loads precomputed `.npy` images using `numpy.load` [41]. Constructor parameters:

- `image_paths`, `labels`: lists of image paths and binary labels.
- `tda_paths` (defaults to `None`): list of paths to precomputed TDA `.npy` files. When provided, `__getitem__` returns a 3-tuple (`image`, `tda`, `label`) instead of (`image`, `label`).
- `use_preprocessing` (defaults to `False`): if `True`, applies the CLAHE + denoise pipeline from `preprocess.py`.
- `augment` (defaults to `False`): if `True`, applies basic augmentation: random horizontal flip via `numpy.fliplr` and random vertical flip via `numpy.flipud`.
- `aggressive_augmentation` (defaults to `False`): if `True`, applies additional augmentation on top of basic flips: random rotation (15) via `cv2.getRotationMatrix2D` and `cv2.warpAffine`, random zoom/crop (85–100% scale), and random shear (0.1).
- `tda_as_image` (defaults to `False`): if `True`, the loaded TDA array is kept as a multi-dimensional array (for `DualViTFusionModel`). If `False`, it is flattened to a 1D vector (for `FusionModel`). item `img_size` (defaults to 224): images are resized to `img_size x img_size` using `cv2.resize`, stacked to 3 channels, normalized to `[0, 1]`, and converted to `torch.Tensor`.

- **src/training/experiment_runner.py**: contains the full training pipeline with four functions:

- `get_class_weights(labels)`: computes the weight for each class for `BCEWithLogitsLoss` to use, as the ratio of negative to positive samples, to address class imbalance in the dataset.
- `train_one_epoch(model, dataloader, optimizer, criterion, device)`: runs one training epoch. Calls `model.train()`, iterates over batches, computes loss, back-propagates, and updates weights. Handles both 2-tuple (ViT-only) and 3-tuple (fusion) batches.

- `evaluate(model, dataloader, criterion, device)`: runs evaluation with `model.eval()` and `torch.no_grad`, to prevent computing the gradient, since `Tensor.backward()` is not used. Computes loss, applies `torch.sigmoid` to get probabilities, and calculates accuracy, F1 score, and AUC-ROC using `scikit-learn` [33]. Returns metrics, true labels, and predicted probabilities.
- `run_experiment(model_fn, train_dataset, test_dataset, config)`: orchestrates a full experiment. Splits training data into train/val using `StratifiedShuffleSplit` (85/15). This guarantees that both classes (benign and malignant) are spread evenly across both groups, the original train group has 48.33% malignant cases and the test one has 51.32%, which is slightly imbalanced, but this study uses the default train-test separation for ease of comparison. Creates `DataLoaders`, initializes the model, loss (with class weights), Adam optimizer) and `CosineAnnealingLR` scheduler. Trains with early stopping (patience-based on validation loss), saves the best model checkpoint using `torch.save` in the `checkpoints` folder, evaluates on the test set, and persists all results (metrics, training history, predictions) to a JSON file using `json.dump` in the `checkpoints` folder, for survival across kernel restarts.
- **src/tda/tda_features.py**: defines six functions for computing topological descriptors from grayscale images:
 - `compute_persistence_image(image)`: uses `CubicalPersistence` [35] to compute persistent homology (tracking connected components and holes across intensity thresholds). It transforms the resulting persistence diagram into a fixed-size `PersistenceImage` [35] of shape (2, 100, 100): one 100x100 image per homology dimension (H0 and H1).
 - `compute_vector_descriptors(image)`: uses `gudhi.CubicalComplex` [36] to compute persistent homology, then extracts compact topological summaries for each homology dimension (H0 and H1) using `BettiCurve`, `Landscape` (5 landscapes), and `Silhouette` [37], all at resolution 100. Returns a 1D numpy array of 1400 dimensions (2×100 Betti + 2×500 Landscape + 2×100 Silhouette).
 - `precompute_tda_cropped_image(cache_csv_path, tda_dir)`: iterates over cached `.npy` crops, computes persistence images via `compute_persistence_image`, saves each as `.npy`, and adds a `tda_crop_path` column to the cache CSV.
 - `precompute_tda_masked_mammography(cache_csv_path, df, tda_dir)`: loads full mammograms and ROI masks from the original `DataFrame`, applies `extract_roi.crop` to produce masked crops, computes persistence images, saves them, and adds a `tda_masked_path` column to the cache CSV.
 - `precompute_tda_vector_descriptors_cropped(cache_csv_path, tda_dir)`: same as `precompute_tda_cropped_image` but uses `compute_vector_descriptors` instead. Adds a `tda_vec_crop_path` column.
 - `precompute_tda_vector_descriptors_masked(cache_csv_path, df, tda_dir)`: same as `precompute_tda_masked_mammography` but uses `compute_vector_descriptors` instead. Adds a `tda_vec_masked_path` column.

- **checkpoints:** contains the files persisted by `run_experiment()` after each experiment: a json file with the results, as well as the best checkpoint obtained, in `pt` format. Each json file contains: test metrics (loss, accuracy, F1, AUC), training history (train loss, val loss, val AUC per epoch), the checkpoint path, and the raw test predictions (`y_true`, `y_prob`) for confusion matrices and ROC curves.

The full code can be found in the zip file attached to this document, or in github: https://github.com/EstherRH00/TDA_Visual_Transformer. The `README.md` file contains all setup and running instructions, and the notebook has been uploaded in html format so that the results are easy to access. All experiments were executed on a single machine with GPU acceleration (CUDA). The random seed is fixed at 2 for all stochastic operations (weight initialization, data splitting, augmentation) to ensure reproducibility.

Chapter 4

Statement on the use of generative artificial intelligence

Generative AI was used; in particular, Claude [42] to do two tasks:

1. Review ortography.
2. Transfer the information from notebook markdown to tex format. Then, it was reviewed by the author.
3. Add documentation to the code in form of function and class comments. It was reviewed by the author before committing.
4. Write the file by file analysis: it was given the `image_utils.py` text as example and was asked to write the rest. Then, it was reviewed by the author.

Chapter 5

Results

This chapter presents the quantitative results obtained from the seven experiments described in Section 3.7. Results are reported on the held-out test set using the best model checkpoint selected by early stopping on the validation set.

Table 5.1 summarizes the test set performance of all experiments in terms of Accuracy, F1 Score, and AUC.

Table 5.1: Test set results for all experiments

Exp.	Description	ACC	F_1	AUC
E1	Baseline ViT (no preprocessing)	0.3995	0.5505	0.5346
E2	ViT + preprocessing	0.6720	0.6196	0.7228
E3	ViT + preprocessing + aggressive aug.	0.6138	0.5101	0.6556
E4	Dual-ViT + PI (cropped lesion)	0.5820	0.3923	0.5687
E5	Dual-ViT + PI (masked mammogram)	0.5926	0.4500	0.5792
E6	Linear fusion + TDA vector (cropped)	0.6243	0.4892	0.6337
E7	Linear fusion + TDA vector (masked)	0.5582	0.5015	0.5787

The best overall performance is achieved by **E2** (ViT with CLAHE + Gaussian denoise, basic augmentation), with an AUC of 0.7228, accuracy of 0.6720, and F1 score of 0.6196.

5.1 Hypothesis 1: Effect of preprocessing

Figure 5.1 shows the comparison of the three experiments addressing H1.

The introduction of preprocessing (E1 $\rightarrow$ E2) produces a substantial improvement across all metrics: AUC increases from 0.5346 to 0.7228 (+35.2% relative), accuracy from 0.3995 to 0.6720, and F1 from 0.5505 to 0.6196.

Adding aggressive augmentation (E2 $\rightarrow$ E3) degrades performance: AUC drops from 0.7228 to 0.6556, accuracy from 0.6720 to 0.6138, and F1 from 0.6196 to 0.5101.

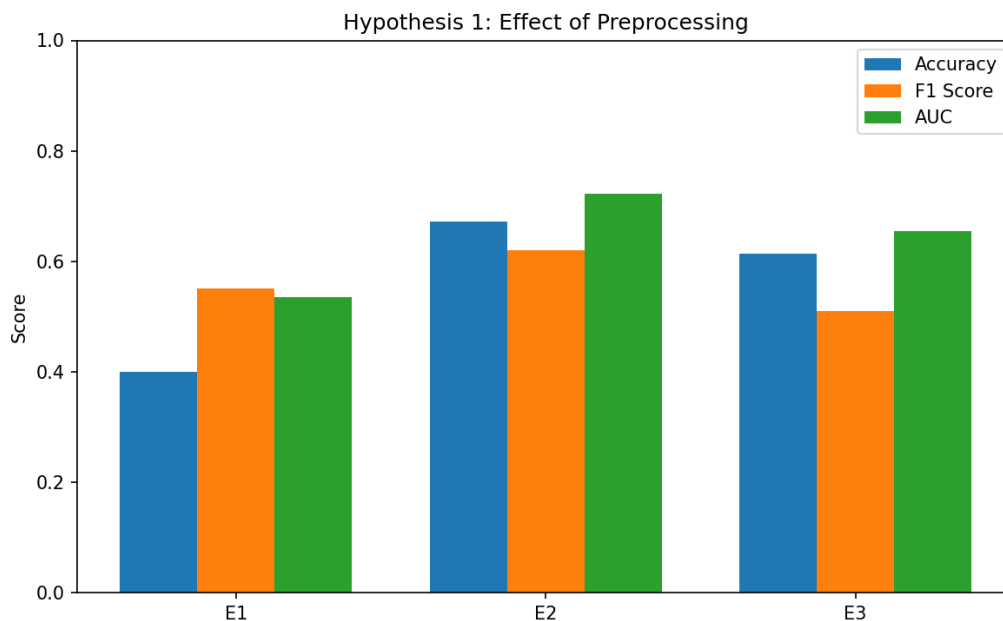


Figure 5.1: AUC, Accuracy, and F1 comparison for experiments E1–E3 (Hypothesis 1).

5.2 Hypothesis 2: Effect of TDA features

Figure 5.2 shows the comparison of the four TDA experiments against the E2 baseline.

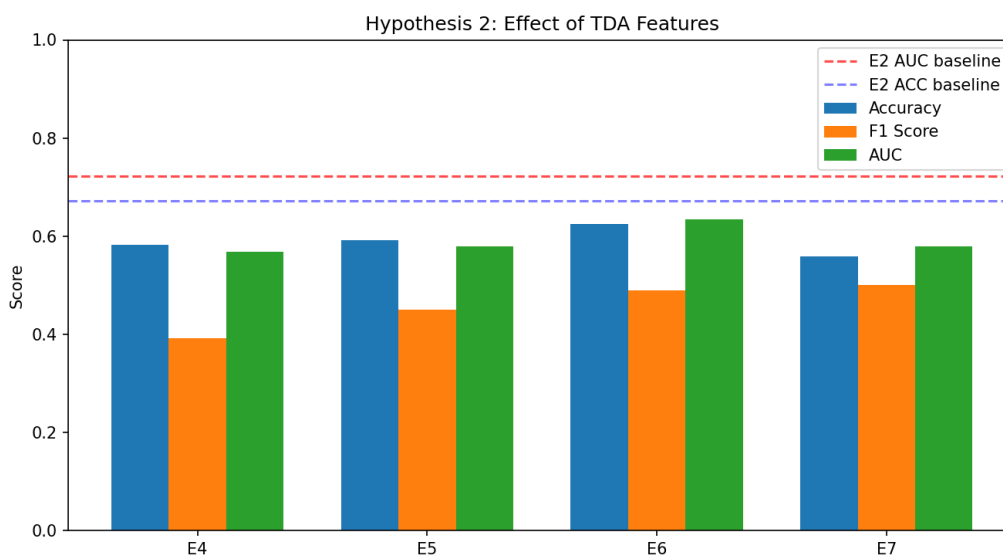


Figure 5.2: AUC, Accuracy, and F1 comparison for experiments E4–E7 (Hypothesis 2) against the E2 baseline (dashed line).

None of the four TDA-enhanced experiments surpasses the E2 baseline:

- The dual-ViT experiments (E4, E5) achieve AUC values of 0.5687 and 0.5792, respectively, well below the 0.7228 baseline.

- The linear fusion experiments (E6, E7) achieve AUC values of 0.6337 and 0.5787, closer to but still below the baseline.
- Among TDA experiments, E6 (linear fusion with cropped lesion vectors) performs best.

5.3 Learning curves

Figure 5.3 shows the training and validation loss curves for all experiments.

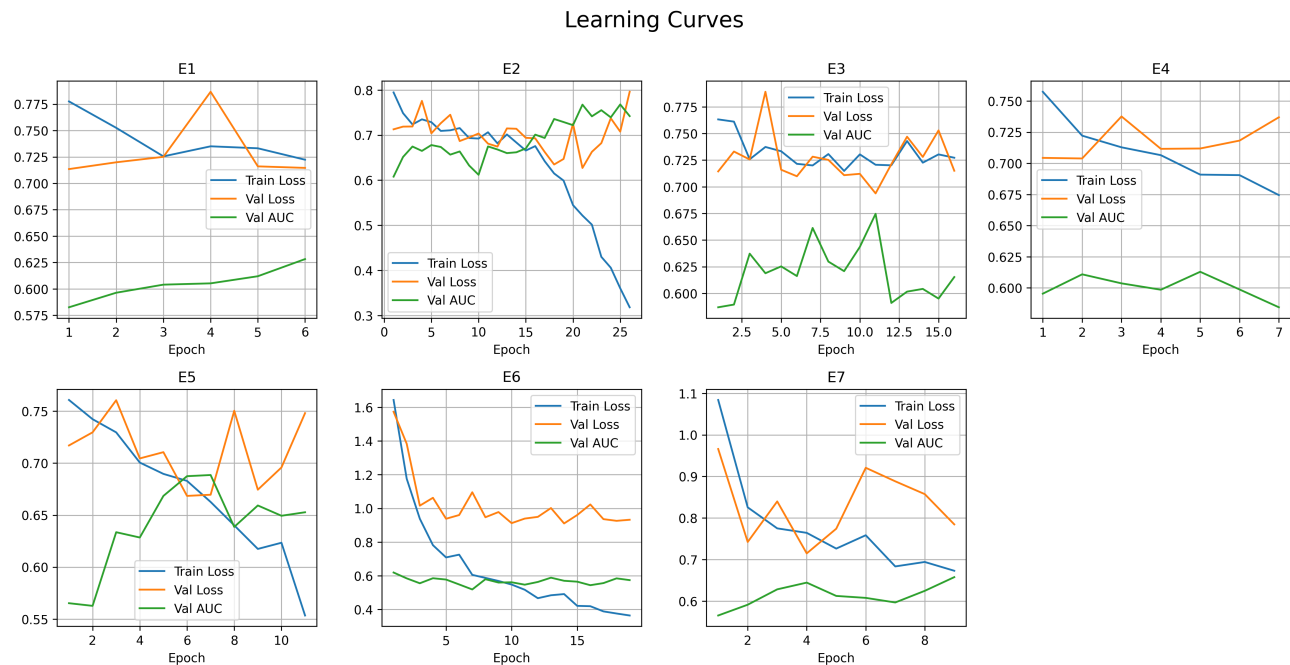


Figure 5.3: Training loss (blue), validation loss (orange) and validation AUC across epochs for all experiments. Early stopping is indicated by the curve termination point.

The learning curves reveal that:

- E1 (no preprocessing) shows minimal learning, with validation loss remaining nearly flat, indicating that the raw images do not provide sufficient signal for the pretrained ViT.
- E2 shows stable convergence with a clear gap between training and validation loss, suggesting moderate overfitting but effective learning.
- E3 (aggressive augmentation) converges more slowly and achieves a higher final validation loss than E2.
- The dual-ViT experiments (E4, E5) exhibit higher variance in validation loss and earlier stopping, consistent with overfitting due to the doubled parameter count.
- The linear fusion experiments (E6, E7) show intermediate behavior between E2 and the dual-ViT models.

5.4 Confusion matrices and ROC curve

Figure 5.4 shows the confusion matrices and figure 5.5 shows ROC curves for all experiments.

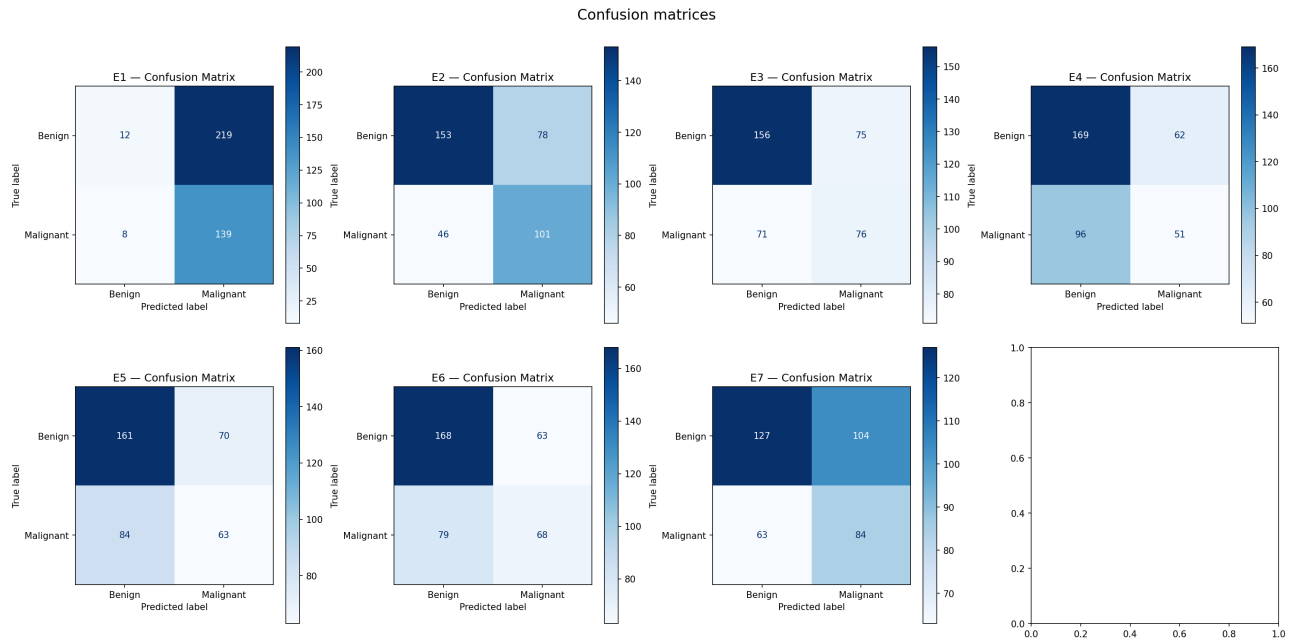


Figure 5.4: Confusion matrices for each experiment

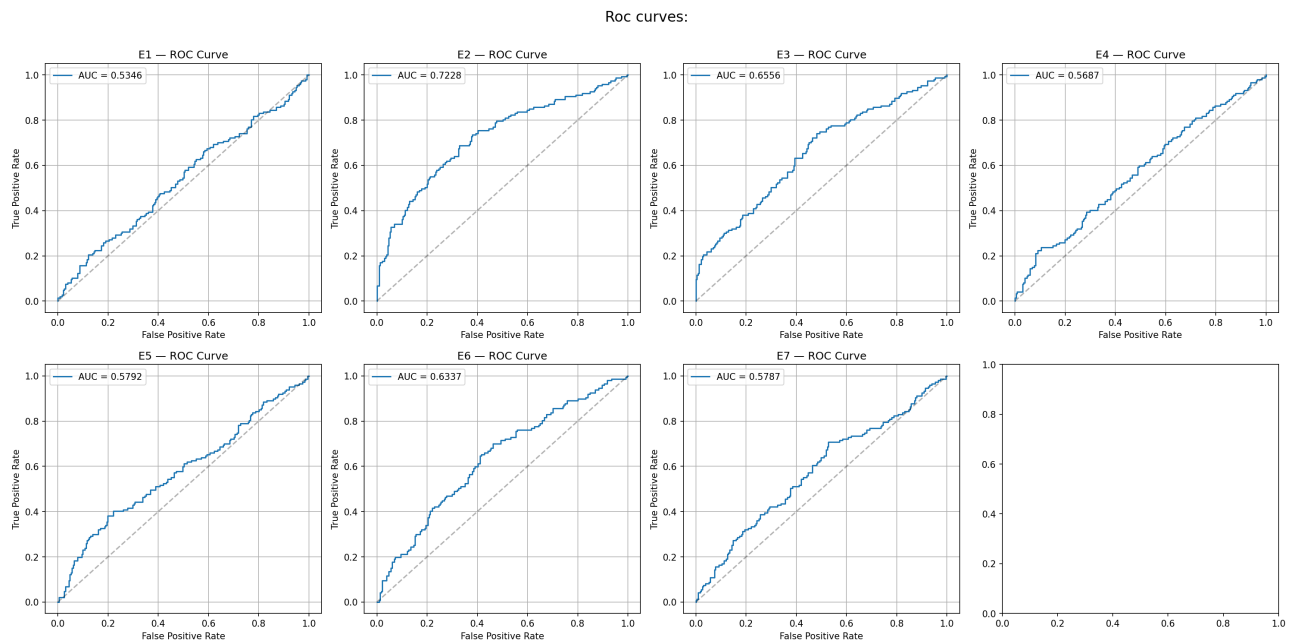


Figure 5.5: ROC curve for each experiment

They show that:

- E1 (no preprocessing) is only slightly better than random, and mostly just predicts "Malign".
- E2 has the better learning, and does predict "Malign" for "Benign" cases with more frequency than the other way around.
- E3 (aggressive augmentation) almost only predicts "Benign" and performs slightly better than E1.
- The dual-ViT experiments (E4, E5) are almost identical to E1, but tend to predict "Benign" instead.
- The linear fusion experiments (E6, E7) show intermediate behavior between E2 and the dual-ViT models. E6 is almost identical to E5 and E7 predicts in a more "balanced" way but fails with almost every "Malign" prediction that does.

5.5 Comparison with the literature

Table 5.2 places the results in context with the studies reviewed in Section 2.1.

Table 5.2: Comparison with CBIS-DDSM binary classification studies from the literature

Study	AUC	ACC	F_1
EfficientNet-B2 (no augmentation) [21]	0.679	0.650	,
Deep BI-RADS Network (BiT + attention + text) [22]	0.878	0.796	0.780
This work: E2 (ViT + preprocessing, ~1K samples)	0.723	0.672	0.620

The best result of this work (E2, $AUC = 0.723$) is comparable to the EfficientNet-B2 study [21] ($AUC = 0.679$), which also used a small dataset without augmentation. The studies achieving higher performance [15, 22] employed substantially larger augmented datasets (60K images) or additional modalities (textual BI-RADS descriptors).

Chapter 6

Discussion

Discussion of results in project context. It is in this section that they make sense and in which the research questions are answered and it is shown how the results respond to the problems raised.

Chapter 7

Conclusions and future works

This chapter must include:

- A description of the conclusions of the work:
 - Once the results have been obtained, what conclusions are drawn?
 - Are these results as expected? Or have they been amazing? Why?
- A critical reflection on the achievement of the objectives initially set:
 - Have we achieved all the objectives? If the answer is negative, why?
- A critical analysis of the monitoring of planning and methodology throughout the product:
 - Has the schedule been followed?
 - Has the planned methodology been adequate enough?
 - Has it been necessary to introduce changes to ensure the success of the work? Why?
- From the impacts foreseen in 1.3, ethical-social, sustainability and diversity, evaluate/mention whether they have been mitigated (if they were negative) or whether they have been achieved (if they were positive).
- If unforeseen impacts have appeared in 1.3, evaluate/mention how they have mitigated (if they were negative) or what they have contributed (if they were positive).
- The future lines of work that have not been explored in this work and have remained pending.

Chapter 8

Glossary

Definition of the most relevant terms and acronyms used in the Report.

Bibliography

- [1] World Health Organization. Cancer, 2024. Accessed: 2026-02-28.
- [2] Mahdie Ahmadi, Nader Karimi, and Shadrokh Samavi. A lightweight deep learning pipeline with drda-net and mobilenet for breast cancer classification, 2024.
- [3] Dr Vijayalakshmi, Binay Pandey, Digvijay Pandey, and Mesfin Esayas. Innovative deep learning classifiers for breast cancer detection through hybrid feature extraction techniques. *Scientific Reports*, 15, 07 2025.
- [4] Pragya Priyadarshini, Shreya Chilumukuru, and Yawo Ezunkpe. Deep learning for early detection of invasive ductal carcinoma in histopathological images: A convolutional neural network approach with transfer learning (preprint). *JMIR Formative Research*, 9, 06 2024.
- [5] Alessandro Carriero, Léon Groenhoff, Elizaveta Vologina, Paola Basile, and Marco Albera. Deep learning in breast cancer imaging: State of the art and recent advancements in early 2024. *Diagnostics*, 14(8), 2024.
- [6] R. Sawyer-Lee, F. Gimenez, A. Hoogi, and D. Rubin. Curated breast imaging subset of digital database for screening mammography (cbis-ddsm) [data set]. <https://doi.org/10.7937/K9/TCIA.2016.7002S9CY>, 2016. The Cancer Imaging Archive.
- [7] U.S. Food and Drug Administration (FDA). Victre breast phantom documentation. <https://breastphantom.readthedocs.io/en/latest/>, 2019. Accessed: 28 February 2026.
- [8] Jawad Ahmad, Sheeraz Akram, Arfan Jaffar, Zulfiqar Ali, Sohail Masood, Awais Ahmad, and Shafiq Ur Rehman. Deep learning empowered breast cancer diagnosis: Advancements in detection and classification. *PLOS ONE*, 19, 07 2024.
- [9] Mohammed Alotaibi, Abdulrhman Aljouie, Najd Alluhaidan, Wasem Qureshi, Hessa Al-matar, Reema Alduhayan, Barrak Alsomaie, and Ahmed Almazroa. Breast cancer classification based on convolutional neural network and image fusion approaches using ultrasound images. *Heliyon*, 9(11):e22406, 2023.
- [10] Fan Wang, Zhilin Zou, Nicole Sakla, Luke Partyka, Nil Rawal, Gagandeep Singh, Wei Zhao, Haibin Ling, Chuan Huang, Prateek Prasanna, and Chao Chen. Topotxr: A topology-guided deep convolutional network for breast parenchyma learning on dce-mris. *Medical Image Analysis*, 99:103373, January 2025.

- [11] Mustafa Tahsn Yilmaz, Enes Algul, and Ishak Pacal. A comparative study of advanced deep learning architectures for breast cancer classification on ultrasound and histological images. *Results in Engineering*, 28:107600, 2025.
- [12] Marina Yusoff, Toto Haryanto, Heru Suhartanto, Wan Mustafa, Jasni Mohamad Zain, and Kusmardi Kusmardi. Accuracy analysis of deep learning methods in breast cancer classification: A structured review. *Diagnostics*, 13:683, 02 2023.
- [13] United Nations. Sustainable development goals. Accessed: 2026-02-28.
- [14] Rebecca Lee, Francisco Gimenez, Assaf Hoogi, Kanae Miyake, Mia Gorovoy, and Daniel Rubin. A curated mammography data set for use in computer-aided detection and diagnosis research. *Scientific Data*, 4:170177, 12 2017.
- [15] Lenin Falconí, Maria Perez, Wilbert Aguilar, and Aura Conci. Transfer learning and fine tuning in breast mammogram abnormalities classification on cbis-ddsm database. *Advances in Science, Technology and Engineering Systems Journal*, 5:154–165, 01 2020.
- [16] Maleika Heenaye-Mamode Khan, Nazmeen Boodoo-Jahangeer, Wasiimah Dullull, Shaista Nathire, Xiaohong Gao, Professor G Sinha, and Kapil Nagwanshi. Multi- class classification of breast cancer abnormalities using deep convolutional neural network (cnn). *PLOS ONE*, 16, 08 2021.
- [17] Pradeep Kumar, K. Sangeetha, A. Sai Praneeth, M. Sethu Madhav, T Swaroopa, and G Rohit. Automatic detection and localization of breast cancer from mammogram imaging modality using modified faster rcnn. In *2023 5th International Conference on Smart Systems and Inventive Technology (ICSSIT)*, pages 1549–1553, 2023.
- [18] Juan Chen, Liangyong Chen, Shengsheng Wang, and Peng Chen. A novel multi-scale adversarial networks for precise segmentation of x-ray breast mass. *IEEE Access*, PP:1–1, 06 2020.
- [19] Akshansh Mishra. Contrast limited adaptive histogram equalization (clahe) approach for enhancement of the microstructures of friction stir welded joints, 2021.
- [20] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition, 2015.
- [21] Vinay V., Akanksha Nayak, and T. Arunkumar. Sensitivity-based breast cancer characterization on cbis-ddsm mammography dataset using fine-tuned efficientnet-b2 model. *International Journal on Advanced Electrical and Computer Engineering*, 15:365–371, 01 2026.
- [22] Gil Ben-Artzi, Feras Daragma, and Shahar Mahpod. Deep bi-rads network for improved cancer detection from mammograms, 2024.
- [23] Samuel J. Magny, Rachel Shikhman, and Ana L. Keppke. *Breast Imaging Reporting and Data System*. StatPearls Publishing, Treasure Island (FL), August 2023. Updated August 28, 2023.

- [24] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021.
- [25] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.
- [26] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus), 2023.
- [27] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015.
- [28] Esther Ruano Hortonedá. Topological data analysis applied to multimodal recommendation, 2024.
- [29] Henry Adams, Sofya Chepushtanova, Tegan Emerson, Eric Hanson, Michael Kirby, Francis Motta, Rachel Neville, Chris Peterson, Patrick Shipman, and Lori Ziegelmeier. Persistence images: A stable vector representation of persistent homology, 2016.
- [30] Shahbaz Hassan, Ahmad Shahid, and Asif Naeem. Tda-vit: A transformer-based framework for unified urdu text recognition via topological and visual feature fusion. *IEEE Access*, PP:1–1, 01 2025.
- [31] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library, 2019.
- [32] Darcy Mason, scaramallion, mrbean bremen, rhaxton, Jonathan Suever, Dimitri Papadopoulos Orfanos, Vanessa Sochat, Guillaume Lemaitre, Chris Bridge, Aditya Panchal, Alex Rothberg, Markus D. Herrmann, Joan Massich, James Kerns, Thomas Robitaille, Simon Biggs, Matthew Shun-Shin, moloney, Blair Conrad, pawelzajdel, Markus Mattes, YoungKi Lyu, Félix C. Morency, Zvi Baratz, Tim Cogan, Taylor, thomsentner, and huicpc0207. pydicom/pydicom: Pydicom v2.4.5, March 2026.
- [33] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [34] G. Bradski. The OpenCV Library. *Dr. Dobb's Journal of Software Tools*, 2000.
- [35] Guillaume Tauzin, Umberto Lupo, Lewis Tunstall, Julian Burella Pérez, Matteo Caorsi, Wojciech Reise, Anibal Medina-Mardones, Alberto Dassatti, and Kathryn Hess. giotto-tda: A topological data analysis toolkit for machine learning and data exploration, 2021.

- [36] Pawel Dlotko. Cubical complex. In *GUDHI User and Reference Manual*. GUDHI Editorial Board, 3.12.0 edition, 2026.
- [37] Pawel Dlotko. Persistence representations. In *GUDHI User and Reference Manual*. GUDHI Editorial Board, 3.12.0 edition, 2026.
- [38] Ross Wightman. Pytorch image models. <https://github.com/rwightman/pytorch-image-models>, 2019.
- [39] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017.
- [40] Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts, 2017.
- [41] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020.
- [42] Anthropic. Claude, 2025. Large language model. Accessed: 2026-05-05.

List of sections that are too extensive to include in memory and have a self-contained character (e.g. user manuals, installation manuals, etc.)

Depending on the type of work, there may be no need to add an annex.